

VENDA.AI - REGISTRO DE CONVERSA E TRANSFERENCIA DE TURNO

Flashinho Coordenador - P0

Data de consolidação: 23/09/2026

Nota de integridade: este PDF consolida o conteúdo disponível nesta conversa e o estado operacional necessário para continuidade. Partes antigas que chegaram ao contexto como trechos resumidos/omitidos pela interface não podem ser reproduzidas literalmente. Segredos, tokens e identificadores pessoais visíveis em capturas foram deliberadamente omitidos.

1. Cadeia de comando e função

Nagila é a autoridade final do Venda.AI. Este chat exerce a função de **Flashinho Coordenador - Venda.AI P0**. A transferência para outro chat não cria funcionário novo: o novo chat assume a mesma função e continua o trabalho.

Frase obrigatória de continuidade: "ISTO NÃO É O INÍCIO DE UM NOVO TRABALHO. É UMA TROCA DE TURNO."

2. Fonte da verdade

Repositório oficial: [ananagila96-lang/venda.ai](https://github.com/ananagila96/lang/venda.ai)

Branch principal: **main**

Constituição: **CONSTITUICAO_MESTRA.md**

Regra: histórico ajuda a preservar conhecimento, mas GitHub determina o estado técnico atual do código.

3. Missão P0 vigente

Fechar o fluxo comercial real ponta a ponta, preservando isolamento por empresa/tenant:

WhatsApp real -> integração -> IA -> conversa registrada -> lead/oportunidade -> GANHO -> validação da venda -> receita validada -> campanha -> ROI

Também deve permanecer funcional o fluxo Cliente -> Conversa -> Histórico e as regras de agenda já aprovadas (confirmação, cancelamento, reagendamento e lista de espera), sem transformar P0 em expansão infinita.

4. Estado do produto consolidado nesta conversa

Área	Estado/decisão
Landing pública	Existe e deve falar com consumidor leigo, não com programador. O objetivo é conduzir Landing -> Cadastro/Contratação -> Área privada.
Área privada	O cliente só deve acessá-la após cadastro/contratação.
Marketing	Existe frente própria; foi solicitado avaliar presença digital mínima necessária ao piloto.
Cliente/Conversa/Histórico	Há histórico de commits e E2E conectando o fluxo; o estado atual deve ser revalidado no GitHub antes de qualquer nova alteração.
Financeiro	Regra central: GANHO não vira receita automaticamente; receita só aparece após validação. ROI deve usar receita validada.
Multiempresa	Cada evento de canal precisa ser associado ao tenant correto antes de alimentar conversa, automação ou resultado.

5. Integração WhatsApp - histórico recente

A equipe testou Z-API, depois Whapi.Cloud, e voltou a decidir pela **Z-API**. Na Whapi houve conexão real do WhatsApp e recebimento comprovado de webhook de mensagem. O payload observado continha campos como messages[], id, from_me, type, chat_id, from, text.body, from_name e channel_id. Esse teste serviu como evidência técnica, mas **a decisão atual é abandonar Whapi e continuar com Z-API**.

Segurança: um token da Whapi apareceu visível em uma captura. Ele deve ser considerado exposto e não reutilizado. Nenhum segredo deve ser colocado em GitHub, PDF, prompt de handoff ou conversa.

Foi criado código de integração Whapi no repositório durante a tentativa. Antes de prosseguir com Z-API, deve-se identificar exatamente as alterações Whapi no estado atual do GitHub e remover/reverter apenas o que for exclusivo dela, preservando qualquer código Z-API anterior e evitando exclusão cega.

6. Decisão vigente: voltar para Z-API

A instrução mais recente de Nagila é: **voltar para a Z-API**. Portanto, o próximo turno deve tratar essa decisão como vigente e não continuar configurando Whapi.

Próximo fluxo-alvo:

WhatsApp real -> Z-API -> backend Venda.AI -> IA -> resposta -> Z-API -> WhatsApp -> persistência -> lead/oportunidade -> venda validada -> ROI

7. Trabalho já realizado / evidências mencionadas

No histórico desta conversa foram mencionados commits como 7093642 (Cliente -> Conversa -> Histórico), 31fb9c8 (E2E desse fluxo) e 0e0d347 (regra financeira campanha/lead/GANHHO/validação/ROI). Também foram mencionados commits posteriores ligados à Whapi. Esses identificadores são **históricos**: o novo turno deve conferir o main atual antes de tratá-los como estado vigente.

8. Regras de execução

PRESERVAR -> ENTENDER -> VERIFICAR -> EXECUTAR -> TESTAR -> DOCUMENTAR -> INTEGRAR.

Não reiniciar o produto. Não duplicar funcionários. Não inventar memória transferida. Não afirmar teste, deploy, commit, integração ou acesso sem evidência. Não apagar código sem entender dependências. Segredos ficam em variáveis de ambiente/secret manager, nunca versionados.

onboarding. Pau no código com evidência. ■

10. Classificação do chat anterior

Após o novo chat confirmar que leu a Constituição, conferiu o GitHub e recebeu este handoff, este chat pode ser classificado como: ■ **ARQUIVAR - SUBSTITUÍDO POR CONTINUAÇÃO.**